

Prática 04 | Guia com código explicado

Pedro Arthur | INF204 | Simulador de votação

O que foi criado?

Uma urna didática com três candidatos. Você informa o mesário, vota em A, B ou C e acompanha os votos, o total e a porcentagem. O botão Zerar Urna limpa somente os votos. É uma simulação para estudar React Native, não uma urna real.

Onde está cada coisa?

A branch do Git é **feature/pratica-04**. A pasta de trabalho é

C:\Users\pedro\Downloads\INF204-aula04\aula04. Dentro dela, o arquivo principal é **App.js**; o PDF que você está lendo é **Guia-Pratica04.pdf**.

A pasta **aula04** que aparece no projeto principal é um atalho para essa pasta de trabalho. Assim, você consegue abrir o código pelo projeto principal, mas o Git continua guardando a Aula 4 na branch certa.

Por que o PDF apareceu fora e dentro?

Durante a montagem, o arquivo foi gerado primeiro em **output\pdf**, uma área temporária de produção. Depois ele foi copiado para **aula04**, que é o local correto para entregar e versionar junto com o código. Agora a cópia que importa para o projeto é a de **aula04\Guia-Pratica04.pdf**.

Como abrir

No terminal, execute **npm install** uma vez e depois **npm start**. Pressione **w** para abrir no navegador. No celular, use o Expo Go na mesma rede Wi-Fi.

1. Estado: os valores que mudam

Estas linhas ficam dentro de **UrnaEletronica**. Cada estado é um pequeno quadro com seu próprio valor e sua própria forma de atualização.

```
const [votosA, setVotosA] = useState(0);
const [votosB, setVotosB] = useState(0);
const [votosC, setVotosC] = useState(0);
const [nomeMesario, setNomeMesario] = useState('');
```

const [valor, alterarValor] separa o valor atual da função que o modifica. **useState(0)** começa um contador em zero; o nome começa vazio. Analogia: cada candidato tem seu próprio placar e o mesário tem seu próprio campo de texto.

Quando um estado muda, o React renderiza a tela de novo. Por isso o número mostrado no placar acompanha cada clique sem precisarmos atualizar o texto manualmente.

2. Props: informações para o componente

O componente **Candidato** é uma peça reutilizável. A urna usa a mesma peça três vezes, entregando dados diferentes para A, B e C.

```
function Candidato({ nome, votos, total, onVotar }) {
  const percentual = total === 0 ? 0 : (votos / total) * 100;
  return (
    <Text>Candidato {nome}: {votos} votos
      ({percentual.toFixed(1)}%)</Text>
  );
}
```

nome, **votos**, **total** e **onVotar** são props: informações recebidas do componente pai. Analogia: a urna entrega uma ficha pronta para o componente, e ele apenas mostra os dados.

A condição **total === 0 ? 0 : ...** evita divisão por zero. Assim a tela mostra 0% no começo, em vez de **NaN**.

3. Callback: somar usando o valor mais recente

Ao apertar um botão, a urna passa uma função para o candidato. O parâmetro **prev** representa o valor anterior mais recente.

```
<Candidato
  nome="A"
  votos={votosA}
  total={totalVotos}
  onVotar={() => setVotosA(prev => prev + 1)}
/>
```

setVotosA(prev => prev + 1) significa: pegue a contagem mais atual e some um. Analogia: pessoas entram numa fila; cada pessoa olha para a última posição e ocupa a próxima. Isso continua seguro mesmo quando os cliques acontecem rapidamente.

O mesmo padrão é usado para B e C. Não se usa um número antigo diretamente, porque atualizações do React podem ser agrupadas.

4. Dados derivados: calcular sem duplicar estado

```
const totalVotos = votosA + votosB + votosC;
const percentual = total === 0 ? 0 : (votos / total) * 100;
```

O total e a porcentagem são calculados a partir dos votos atuais. Eles não precisam de outro **useState**.

Analogia: uma calculadora soma os placares quando precisa; não é necessário guardar uma segunda folha com a mesma informação.

5. Campo controlado do mesário

```
<TextInput
  value={nomeMesario}
  onChangeText={setNomeMesario}
  placeholder="Digite seu nome"
/>
```

value mostra o estado no campo. **onChangeText** atualiza o estado a cada tecla. Analogia: o texto digitado e o quadro de controle são um espelho: um sempre reflete o outro.

6. Reset: zerar somente os votos

```
const zerarUrna = () => {  
  setVotosA(0);  
  setVotosB(0);  
  setVotosC(0);  
};
```

A função coloca os três contadores em zero. O nome do mesário não é alterado, porque ele é outro estado e não faz parte da contagem. Analogia: limpar o placar não apaga o nome de quem está trabalhando.

```
<TouchableOpacity onPress={zerarUrna}>  
  <Text>Zerar Urna</Text>  
</TouchableOpacity>
```

onPress diz qual função deve ser executada quando o botão for pressionado.

7. A tela juntando as peças

```
<Text>Mesário atual: {nomeMesario || 'Não informado'}</Text>  
<Candidato nome="A" votos={votosA} total={totalVotos}  
  onVotar={() => setVotosA(prev => prev + 1)} />  
<Text>Total de Votos: {totalVotos}</Text>
```

As chaves `{ }` colocam valores JavaScript dentro do JSX. É como colocar etiquetas em um painel: cada etiqueta recebe o valor atual do estado.

8. Como conferir funcionando

Digite Pedro Arthur no campo. Vote duas vezes em A e uma em B: o total deve ser 3, A deve mostrar 66,7% e B 33,3%. Clique em Zerar Urna: os votos voltam a zero e o nome permanece.

No terminal, **npm test** confere a lógica automaticamente. **npx expo export --platform web** verifica se o aplicativo consegue ser compilado para a web.

Arquivos principais

App.js contém a tela e a lógica. **package.json** contém comandos e dependências. **app.json** configura o Expo. **tests/urna.test.cjs** verifica o comportamento. **.gitignore** evita enviar caches como `node_modules`.